Lab Assignment on Trigger

DBMS_Assignment 6 _PCA2_3_2026

Note:

Solutions for 1 & 2 are given…do the rest on your own.
Tables required for each query are given at the end.

1. BEFORE INSERT Trigger on Employee Table

Create a trigger on the EMPLOYEE table that checks whether the salary entered is less than 5000. If yes, automatically set it to 5000 before insertion.

Solution:

MySQL

```
CREATE TABLE EMPLOYEE (
   EMP_ID INT,
   EMP_NAME VARCHAR(50),
   SALARY INT
);

DELIMITER //

CREATE TRIGGER trg_salary_check
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
   IF NEW.SALARY < 5000 THEN
      SET NEW.SALARY = 5000;
   END IF;
END //
```

Oracle SQL

```
CREATE TABLE EMPLOYEE (
   EMP_ID NUMBER,
   EMP_NAME VARCHAR2(50),
   SALARY NUMBER
);

CREATE OR REPLACE TRIGGER trg_salary_check
BEFORE INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
   IF :NEW.SALARY < 5000 THEN
      :NEW.SALARY := 5000;
   END IF;
```

```
END;
/
```

2. AFTER INSERT Trigger for Audit Table

Create an AUDIT_EMPLOYEE table to store employee insert details. Write a trigger so that whenever a new employee is inserted into EMPLOYEE, the employee ID, name, and insertion date are stored in the audit table.

MySQL

```
CREATE TABLE AUDIT_EMPLOYEE (
   EMP_ID INT,
   EMP_NAME VARCHAR(50),
   INSERT_DATE DATE
);

DELIMITER //

CREATE TRIGGER trg_employee_audit
AFTER INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
   INSERT INTO AUDIT_EMPLOYEE
   VALUES (NEW.EMP_ID, NEW.EMP_NAME, CURDATE());
END //

DELIMITER ;
```

Oracle SQL

```
CREATE TABLE AUDIT_EMPLOYEE (
   EMP_ID NUMBER,
   EMP_NAME VARCHAR2(50),
   INSERT_DATE DATE
);

CREATE OR REPLACE TRIGGER trg_employee_audit
AFTER INSERT ON EMPLOYEE
FOR EACH ROW
BEGIN
   INSERT INTO AUDIT_EMPLOYEE
   VALUES (:NEW.EMP_ID, :NEW.EMP_NAME, SYSDATE);
END;
/
```

3. BEFORE DELETE Trigger on Student Table

Create a trigger on the STUDENT table that prevents deletion of records of students whose marks are greater than 80.

4. AFTER UPDATE Trigger for Salary Change

Create a trigger on the EMPLOYEE table that stores old salary, new salary, and update date in a separate table called SALARY_LOG whenever salary is updated.

5. BEFORE UPDATE Trigger on Salary Table

Create a trigger on the ENPLOEE table to ensure that Salary cannot be increased by more than 25% during an update.

6. AFTER DELETE Trigger on Customer Table

Create a trigger that stores deleted customer details in a table called CUSTOMER_BACKUP whenever a record is deleted from the CUSTOMER table.

7. BEFORE INSERT Trigger on Attendance Table

Create a trigger on the ATTENDANCE table to automatically set attendance status as 'Absent' if no status is provided during insertion.

8. AFTER INSERT Trigger on Student Table

Whenever a new student is inserted into the STUDENT table, automatically **increase the student count** in the corresponding DEPARTMENT table.

9. BEFORE INSERT Trigger for Age Validation

Create a trigger on the STUDENT table that does not allow insertion if the student's age is less than 17.

10. AFTER UPDATE Trigger on Department Table

Create a trigger that records changes made to the department name in a DEPARTMENT_LOG table whenever the department name is updated.

Tables and Their Attribute Names (1–8)

1. **EMPLOYEE** → EMP_ID, EMP_NAME, SALARY
2. **EMPLOYEE, AUDIT_EMPLOYEE** →
   EMPLOYEE: EMP_ID, EMP_NAME, SALARY
   AUDIT_EMPLOYEE: EMP_ID, EMP_NAME, INSERT_DATE
3. **STUDENT** → STUD_ID, STUD_NAME, MARKS
4. **EMPLOYEE, SALARY_LOG** →
   EMPLOYEE: EMP_ID, EMP_NAME, SALARY
   SALARY_LOG: EMP_ID, OLD_SALARY, NEW_SALARY, UPDATE_DATE
5. EMPLOYEE: EMP_ID, EMP_NAME, SALARY

6. **CUSTOMER, CUSTOMER_BACKUP** →
   CUSTOMER: CUST_ID, CUST_NAME, CITY
   CUSTOMER_BACKUP: CUST_ID, CUST_NAME, CITY
7. **ATTENDANCE** → ATT_ID, STUD_ID, ATT_DATE, STATUS
8. **STUDENT, DEPARTMENT** →
   STUDENT: STUD_ID, STUD_NAME, AGE, DEPT_ID
   DEPARTMENT: DEPT_ID, DEPT_NAME, STUDENT_COUNT